

Algoritmo de AdaBoosting

Parte de la familia de Métodos de Consenso y Potenciación

Machine Learning con Python Avanzado
Lead University

April 28, 2026

Agenda de la presentación

1. Introducción
2. Bootstrap
3. Métodos de Consenso y Potenciación
4. AdaBoosting
5. Demostracion

Table of Contents

- 1 **Introducción**
- 2 Bootstrap
- 3 Métodos de Consenso y Potenciación
- 4 AdaBoosting
- 5 Demostración

Inicio del "boosting"

- El algoritmo de boosting nace del cuestionamiento de Kearns y Valiant (1988 y 1989), sobre si se podía crear un **clasificador robusto** a partir de **clasificadores débiles**.
- Al comienzo se llamaba "boosting" al proceso de convertir un clasificador débil en uno robusto.
- Robert Schapire (1990) contesta la interrogante planteada y desencadena una "revolución" en esta área de conocimiento.
- Los primeros algoritmos (Schapire y Freund) no se "adaptaban" a los errores y le sacaban mayor provecho. Posteriormente desarrollaron el "*Adaptive Boosting*" y ganaron el Premio Gödel.

Los siguientes modelos, y todas sus variaciones, se fundamentan en el descenso de gradiente para realizar el análisis.

- LPBoost
- TotalBoost
- BrownBoost
- XGBoost
- MadaBoost
- LogitBoost

Ventajas

- La precisión de los clasificadores débiles se puede mejorar usando Adaboost.
- Es menos propenso al sobreajuste ya que los parámetros de entrada no se optimizan conjuntamente.
- La flexibilidad le permite combinarse con cualquier tipo de algoritmo de aprendizaje.
- No requiere conocimiento previo para los clasificadores.
- Versatilidad de aplicaciones.

Desventajas

- Extremadamente sensible a los datos ruidosos y los valores atípicos, esto porque el algoritmo podría asignar pesos a ciertos individuos que no son representativos.
- Se ha demostrado que AdaBoost es más lento que XGBoost.
- Necesidad de un *data set* limpio (sin valores extremos o atípicos).
- Limitación de escala.

Introducción - aplicaciones

Algunos usos

- Clasificación de observaciones
- Reconocimiento de personas
- Reconocimiento de emociones
- Identificación de riesgos
- Aprendizaje

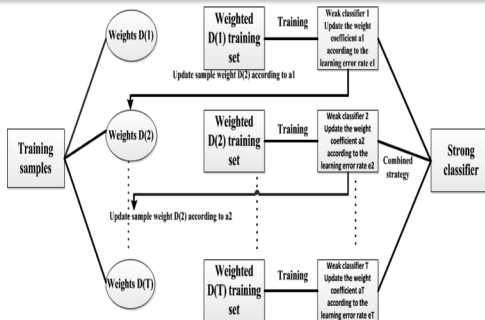


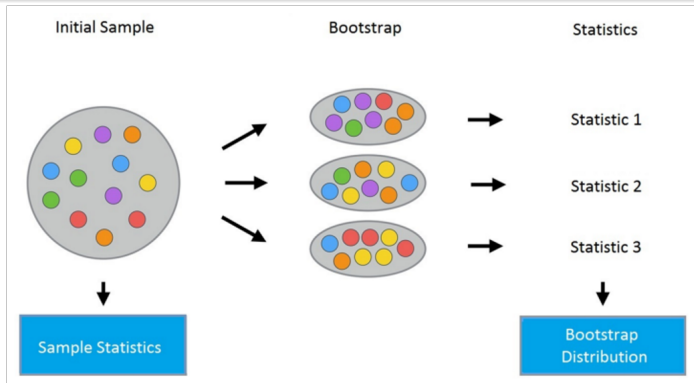
Table of Contents

- 1 Introducción
- 2 Bootstrap**
- 3 Métodos de Consenso y Potenciación
- 4 AdaBoosting
- 5 Demostración

Bootstrapping

¿Qué es Bootstrapping?

Es un método estadístico de remuestreo propuesto por Bradley Efron en 1979. En vez de repetidamente tomar observaciones independientes y crear *data sets* de la población, se muestrea repetidamente el *data set* original y se usan esas observaciones.

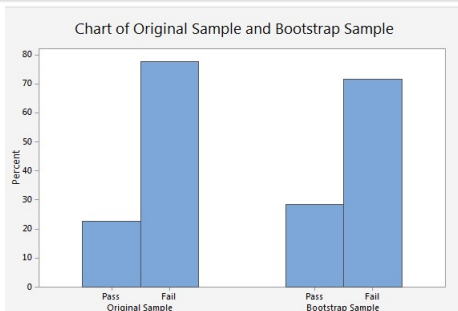


Bootstrapping - Teorema

Teorema a considerar

Teorema: En promedio, una muestra de arranque D_i (*bootstrap*) contiene aproximadamente 63% de los datos de entrenamiento originales.

Prueba: Cada individuo tiene una probabilidad de $1 - (1 - 1/n)^n$ de ser seleccionado en cada D_i . Si n es suficientemente grande, esta probabilidad converge a $1 - \frac{1}{e} = 0.632$.



Bootstrapping - Teorema (Construcción)

De una población $\mathcal{M}$, la probabilidad de ser escogido es $\frac{1}{n}$, donde n es el tamaño de M . Por *bootstrapping*, se realiza esa selección **con remplazo** la cantidad de veces que sea necesario.

$$B1_1 = \left(1 - \frac{1}{n}\right), \dots, B1_n = \left(1 - \frac{1}{n}\right) \rightarrow \text{Probabilidad de no ser elegido en el } B1_i.$$

Como son eventos independientes, la probabilidad es

$$\prod_{i=1}^n \left(1 - \frac{1}{n}\right) = \left(1 - \frac{1}{n}\right)^n \rightarrow \text{Probabilidad de no ser elegido en el } B1.$$

La probabilidad de **ser elegido** en el *bootstrap* es el complemento de lo anterior.

$$1 - \left(1 - \frac{1}{n}\right)^n$$

Bootstrapping - Teorema (Construcción)

Para entender el comportamiento asintótico se toma el $\lim_{n \rightarrow \infty}$, pero se necesita una sustitución de la forma $m = \frac{1}{n}$ que tiene $\lim n \rightarrow \infty = \infty$ y $\lim m \rightarrow \infty = 0$

$$\lim_{n \rightarrow \infty} 1 - \left(1 - \frac{1}{n}\right)^n \implies \lim_{m \rightarrow \infty} 1 - (1 - m)^{\frac{1}{m}}$$

Para eliminar el exponente, se aplica un logaritmo y se utiliza la propiedad del exponente.

$$\lim_{m \rightarrow \infty} \frac{1}{m} \ln(1 - m)$$

Objetivo del boosting

- “Boosting” es un método general para mejorar el rendimiento de cualquier algoritmo de aprendizaje.
- Reduce significativamente el error de cualquier algoritmo de aprendizaje “débil”.

Table of Contents

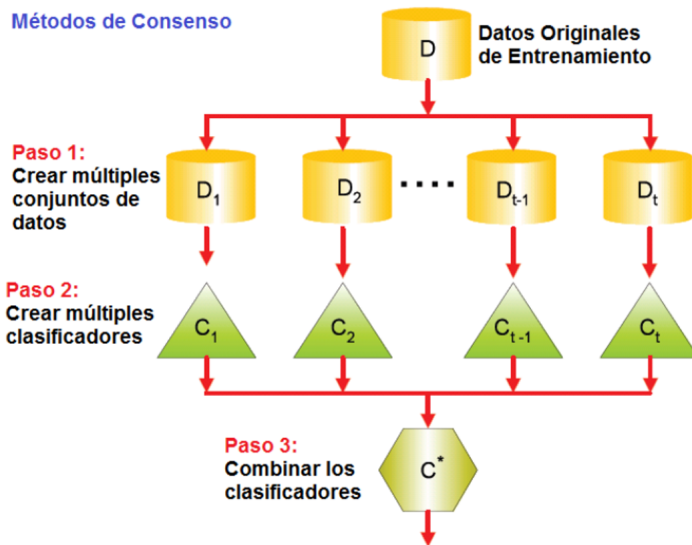
- 1 Introducción
- 2 Bootstrap
- 3 Métodos de Consenso y Potenciación**
- 4 AdaBoosting
- 5 Demostracion

¿Qué son los métodos de consenso?

- La idea es tomar m muestras aleatorias con *Bootstrap* de los datos originales y luego aplicar en cada una de ellas un método predictivo.
- Dados los resultados de cada modelo, gana el que “mayor cantidad de votos” tenga.

Métodos de consenso y potenciación

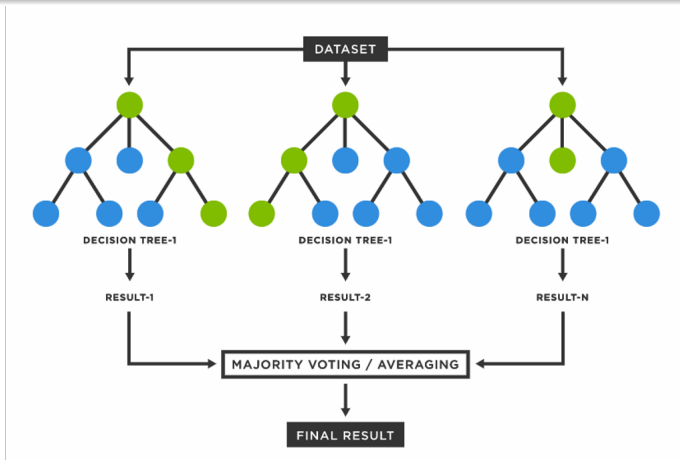
Métodos de Consenso



Métodos de consenso y potenciación

Bosques Aleatorios

El caso en el que todos los clasificadores del Método de Consenso son Árboles dicho método se denomina Bosques Aleatorios (*Random Forest*).

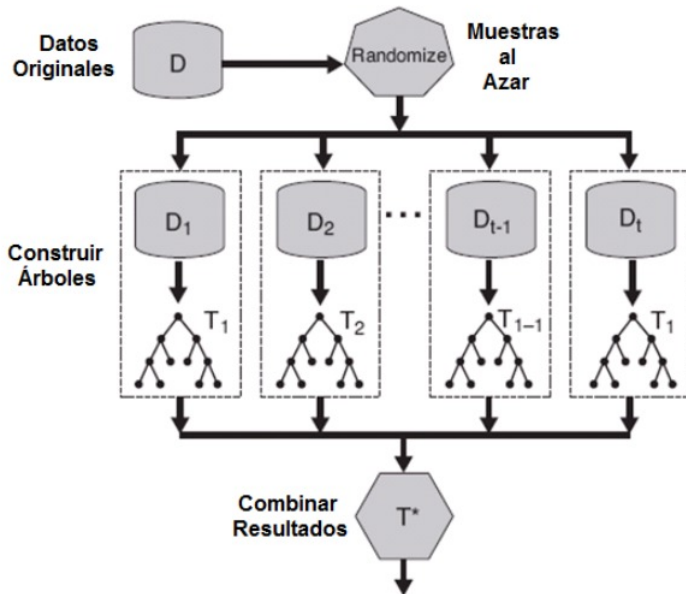


Métodos de consenso y potenciación

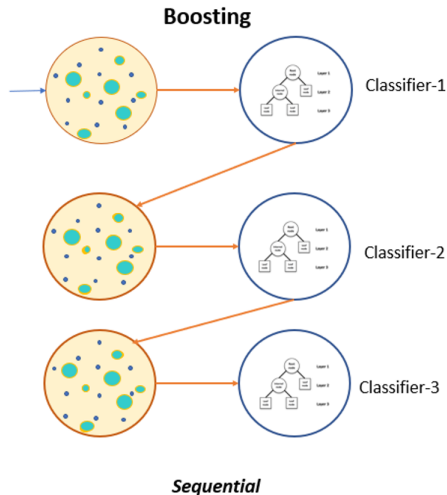
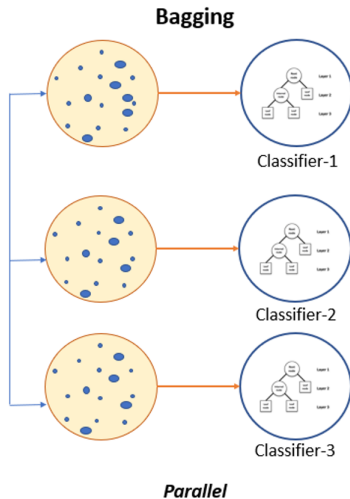
Objetivo de AdaBoost

La idea es tomar una muestra aleatoria de los datos originales y aplicar sobre ésta un método clasificadorio para luego aumentar el peso (potenciar) a los individuos que estén mal clasificados para la siguiente clasificación tengan prioridad.

Métodos de consenso y potenciación - Ejemplo



Bootstrapping - Bagging¹ y Boosting



¹Bootstrapping aggregating

Table of Contents

- 1 Introducción
- 2 Bootstrap
- 3 Métodos de Consenso y Potenciación
- 4 AdaBoosting**
- 5 Demostracion

Orígenes

- 1 L. Breiman y J. Friedman (investigadores de la Universidad de Stanford) han trabajado y potenciado el modelo de árboles de decisión (Random Forest).
- 2 Freund y Schapire en 1997 propusieron el primer algoritmo de la familia "Boosting" - bautizado como "AdaBoost.M1".
- 3 Boosting es una de las ideas de aprendizaje supervisado más poderosas introducidas en los últimos veinte años. (*Hastie, T.*)
- 4 Leo Brieman coronó a AdaBoost (Freund y Schapire, 1996) (el primer algoritmo de Boosting) como el "mejor clasificador estándar del mundo".

The Annals of Statistics
2004, Vol. 32, No. 1, 1–11
© Institute of Mathematical Statistics, 2004

THE 2002 WALD MEMORIAL LECTURES POPULATION THEORY FOR BOOSTING ENSEMBLES

BY LEO BREIMAN

University of California, Berkeley

Tree ensembles are looked at in distribution space, that is, the limit case of “infinite” sample size. It is shown that the simplest kind of trees is complete in D -dimensional $L_2(P)$ space if the number of terminal nodes T is greater than D . For such trees we show that the AdaBoost algorithm gives an ensemble converging to the Bayes risk.

1. Introduction. Using ensembles of predictors for classification and regression has proved to give more accurate results than use of a single predictor. The most well-known ensemble method is the AdaBoost algorithm, introduced by Freund and Schapire [12]. Complex in form, it was originally designed to drive the training set error rapidly to zero. But, surprisingly, empirical experiments showed that the generalization error kept decreasing long after the training set error was zero—in fact, slowly decreasing as hundreds of trees were added to the ensemble.

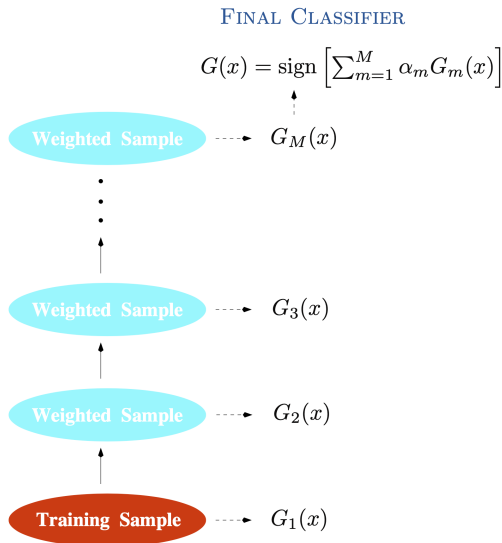
After its introduction, AdaBoost became known as the most accurate general purpose classification algorithm available. (For performance benchmarks, see [1, 8, 9].) Hundreds of papers covering various modifications of AdaBoost or applications of AdaBoost were and are being published in the Machine Learning literature. But the understanding of how and why AdaBoost worked was lacking until a short time ago. During the intervening years I often referred, in prepared talks and conversations, to the mystery of AdaBoost as the most important unsolved problem in Machine Learning.

Schapire, Freund, Bartlett and Lee [17] constructed a bound on the generalization error of ensembles in terms of the training set margin distribution and the VC-dimension of the individual predictors in the ensemble. This led to the conjecture that the key to the low generalization error of the AdaBoost algorithm was that it produced a high margin distribution. But Breiman [5] constructed an algorithm that on a variety of data sets produced uniformly higher margin distributions than AdaBoost using predictors with the same VC-dimension, yet had higher generalization error.

A significant discovery, due to Breiman [3] (and rediscovered several times afterward) was that AdaBoost was a down-the-gradient method for minimizing an exponential function of the error (see Section 3). Then “boosting” became a generic term for greedy down-the-gradient minimization of various loss functions.

Received August 2002; revised May 2003.
AMS 2000 subject classifications: 62H30, 68T10, 68T05.
Key words and phrases: Trees, AdaBoost, Bayes risk.

AdaBoosting



Dinámica de pesos Adaboost.M1

Round	1	2
1	0.1	0.311
2	0.1	0.311
3	0.1	0.311
4	0.1	0.01
5	0.1	0.01
6	0.1	0.01
7	0.1	0.01
8	0.1	0.01
9	0.1	0.01
10	0.1	0.01

Aspecto relevante

Las predicciones de todos ellos luego se combinan a través de un voto de mayoría ponderada para producir la predicción final:

$$G(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right) \quad (1)$$

Algoritmo de AdaBoosting

AdaBoost.M1

- ➊ Inicializar los pesos $w_i = 1/N, i = 1, 2, \dots, N$.
- ➋ Para $m = 1$ a M :
 - ➊ Ajustar el clasificador $G_m(x)$ a los datos de prueba usando w_i .
 - ➋ Calcular:

$$\text{err}_m = \frac{\sum_{i=1}^N w_i \cdot I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i} \quad (2)$$

- ➌ Calcular $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$
 - ➍ Establecer $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))], i = 1, 2, \dots, N$
- ➌ Resultado

$$G(x) = \text{sign} \left[\sum_{m=1}^M \alpha_m G_m(x) \right] \quad (3)$$

Table of Contents

- 1 Introducción
- 2 Bootstrap
- 3 Métodos de Consenso y Potenciación
- 4 AdaBoosting
- 5 Demostración**

AdaBoosting en Python

Código

```
class sklearn.ensemble.AdaBoostClassifier(args)
```

- `base_estimator = None`
- `n_estimators = 50`
- `learning_rate = 1.0`
- `algorithm = SAMME.R`
- `random_state = None`